

Suitably Impressive Thesis Title



Candidate Number: 1234567

University of Oxford

Word Count: 15000

A thesis submitted for the degree of

Master of Science in Advanced Computer Science

Trinity 2026

Abstract

Your abstract goes here. Check your departmental regulations, but this is usually expected to be under 300 words — roughly one page.

A workable structure is one paragraph per move: (1) context and why the problem matters, (2) the gap, (3) what you did, (4) what you found, and (5) what it means more broadly.

Acknowledgements

Thank your supervisors first, then anyone whose ideas or feedback shaped the direction of the work.

Then colleagues, collaborators, and friends.

Then any funding or institutional support you received.

Finally, family.

Contents

1	Introduction	1
1.1	Problem Setting and Research Gap	2
1.2	Proposed Approach and Main Findings	3
1.3	Contributions	4
1.4	Thesis Overview	5
2	Background	6
2.1	Generative Modelling	6
2.1.1	Conditional Generative Modelling	7
2.1.2	Inverse Problems	7
2.1.3	Relation to Reward Alignment	8
2.2	Flow-Based Generative Modelling	9
2.2.1	Normalising Flows	9
	Bibliography	16
A	Supplementary Material	20
A.1	Change of Variables	20
A.2	Score functions and score matching	20
A.3	Additional Experiments	20
A.3.1	GMM Experiments	20

List of Figures

Chapter 1

Introduction

One goal of generative modelling is to learn how to produce novel samples that resemble those drawn from an unknown data distribution p_{data} . Diffusion and flow models [1–5] have been highly successful at this task, achieving state-of-the-art results in image, video and audio generation [6–8]. Both use the same high-level framework for generation: samples $\mathbf{z} \sim p_{\text{base}}$ drawn from a simple noise distribution are transformed continuously over time towards the data distribution [9].

Flow and diffusion models specify the dynamics of this continuous transformation between noise and data as governed by an ordinary differential equation (ODE) and a stochastic differential equation (SDE), respectively [4, 10]. An ODE defines a path from a noise to data sample by specifying the local velocity at each point along it. In a flow model, a neural network is trained to predict this velocity. Generating data therefore consists of iteratively advancing a sample from noise to data through a sequence of small steps, querying the neural network at each step to determine the direction of travel. Consequently, although flow models enable high-fidelity generation, they can often require hundreds of neural network evaluations [3], which can be computationally prohibitive.

Flow maps [11, 12] address this limitation by training a neural network to predict the cumulative displacement between any two times along the noise-to-data paths defined by the ODE, rather than the local velocity at a single point. A flow map can therefore transport a noise sample directly to a data sample in a single neural network evaluation. Flow maps can be learned by bootstrapping from a pre-trained flow model via distillation [11, 13]. However, it is also possible to train a flow map from scratch, as demonstrated by MeanFlow [14].

1.1 Problem Setting and Research Gap

Many applications require conditional generation: producing samples from the data distribution that satisfy an external condition, such as a class label or text prompt [6, 15, 16]. A canonical example of conditional generation is inverse problem solving [17], in which an observation $\mathbf{y}$ is generated by corrupting an unknown data sample $\mathbf{x} \sim p_{\text{data}}$ according to $\mathbf{y} = \mathcal{A}(\mathbf{x}) + \boldsymbol{\eta}$, where $\mathcal{A}$ is a known forward operator and $\boldsymbol{\eta}$ represents measurement noise. In image inverse problems, for example, $\mathcal{A}$ may downsample, blur or mask an image. Since these operations typically discard information, multiple data samples may be consistent with the same observation. The goal of inverse problem solving is therefore to draw samples from the posterior $p_{\text{data}}(\mathbf{x} \mid \mathbf{y})$, which represents the distribution of plausible data samples consistent with the given observation $\mathbf{y}$.

One approach to conditional generation is to train a generative model specifically for the conditional task, either from scratch or by fine-tuning a pretrained unconditional model [16, 18]. This can provide fast conditional generation, including one-step generation with a flow map [14, 19]. However, adapting the generative model to each new conditional generation setting typically requires a new round of training or fine-tuning, which can be prohibitively expensive for large generative models.

Guidance-based methods offer an alternative for flow (and diffusion) models. These methods keep the pretrained unconditional flow model frozen but nudge its velocity prediction at each sampling step, thereby steering the path from noise towards a data sample that satisfies the given condition [20–23]. This approach hence allows for same unconditional model to be reused across different conditional generation settings without fine-tuning.

However, this guidance strategy does not transfer naturally to one-step generation with a flow map. A flow map transports an initial noise sample directly to a data sample in a single evaluation, leaving no intermediate predictions through which guidance can be applied. The key insight of Variational Flow Maps (VFM) [24] is that generation can instead be controlled through the choice of starting noise. For a one-step unconditional flow map, the starting noise is the only remaining lever for controlling its output. Conditional generation can therefore be reframed as learning a distribution over starting noise samples that the flow map sends to data samples satisfying the given condition.

Let $p_{\theta}(\mathbf{x})$ denote the model distribution (which approximates $p_{\text{data}}(\mathbf{x})$) obtained by drawing $\mathbf{z} \sim p_{\text{base}}$ and passing it through the flow map, where θ denotes

the neural-network parameters of the flow map. For a given condition $\mathbf{y}$, VFM formulates the required noise distribution as the noise-space posterior $p_\theta(\mathbf{z} \mid \mathbf{y})$. Intuitively, this distribution is obtained by pulling the corresponding model posterior $p_\theta(\mathbf{x} \mid \mathbf{y})$ backwards through the flow map into noise space.

Although $p_\theta(\mathbf{z} \mid \mathbf{y})$ cannot be sampled from directly, its density can be evaluated up to a normalising constant, making variational inference (VI) [25] a natural approach for approximating it. VI introduces a tractable distribution $q_\phi(\mathbf{z} \mid \mathbf{y})$, called a variational distribution, with parameters ϕ , to approximate this posterior. Fitting $q_\phi(\mathbf{z} \mid \mathbf{y})$ using the variational objective requires both being able to sample from it and evaluate its density. VFM satisfies these requirements by parameterising $q_\phi(\mathbf{z} \mid \mathbf{y})$ as a diagonal Gaussian whose mean and variance are predicted from $\mathbf{y}$ by a neural network, termed the noise adapter.

Although a Gaussian supports both sampling and exact density evaluation, it may be too simple to approximate the potentially complex noise-space posterior $p_\theta(\mathbf{z} \mid \mathbf{y})$. VFM compensates for this by jointly training the noise adapter and the flow map, allowing the geometry of the noise space to be reshaped such that $p_\theta(\mathbf{z} \mid \mathbf{y})$ can be more easily approximated by a simple Gaussian distribution. VFM therefore enables one-step conditional generation, but only by fine-tuning the flow map. This leaves open whether one-step conditional generation can be achieved while keeping the unconditional flow map fixed.

1.2 Proposed Approach and Main Findings

The central question of this project is whether a more expressive variational distribution $q_\phi(\mathbf{z} \mid \mathbf{y})$ can approximate $p_\theta(\mathbf{z} \mid \mathbf{y})$ while leaving the pretrained flow map frozen. We investigate this question by replacing VFM’s Gaussian noise adapter with a conditional normalising flow [26, 27]. Normalising flows support both sampling and exact density evaluation, satisfying the requirements of variational training while being capable of representing substantially more complex distributions than a Gaussian. When amortising over observations $\mathbf{y}$, the inverse-problem setting also naturally permits the approximating distribution, $q_\phi(\mathbf{z} \mid \mathbf{y})$, to be learned via maximum likelihood estimation. We therefore introduce a training objective that combines the variational loss with a maximum-likelihood loss [28], and show that the combination is more effective than either loss alone.

The resulting approach enables one-step conditional generation using an

existing pretrained unconditional flow map while keeping its parameters fixed. Solving each new conditional generation setting using this approach requires training only a lightweight noise adapter. We evaluate the method across inverse problems of increasing difficulty, progressing from a two-dimensional checkerboard problem to image brightness, $4\times$ super-resolution and inpainting. On the two-dimensional problem, the Gaussian adapter fails to approximate the noise-space posterior accurately, whereas the normalising-flow adapter captures its multimodal structure. For $4\times$ super-resolution, our method achieves the lowest maximum mean discrepancy (MMD) among all evaluated baselines, including VFM.

However, we find that the approach is not competitive on image inpainting. Our analysis suggests that the main difficulty is learning, in a single step, which initial noise samples will map to images matching an observation that constrains many pixel values. This motivates training the adapter to predict a point partway along the noise-to-data path, rather than the initial noise itself, before mapping that point to data in a single flow-map evaluation. We leave the development of this extension to future work.

1.3 Contributions

This thesis makes the following contributions:

1. We adapt the Variational Flow Maps (VFM) framework for one-step conditional generation so that it is training-free with respect to the flow map. In particular we replace the Gaussian noise adapter with a more expressive normalising flow. To our knowledge, this is the first work to investigate solving conditional generation entirely within the noise space of a pretrained unconditional flow map without fine-tuning the flow map itself.
2. We further extend VFM by introducing a training objective that combines both the variational loss with a maximum-likelihood loss. We show that this combined objective is more effective than training the noise adapter with either maximum likelihood or the variational loss alone.
3. We introduce a principled evaluation framework that isolates error attributable to the trained noise adapter from error attributable to the frozen pretrained flow map.

4. We demonstrate that the more expressive normalising flow adapter can substitute for flow map fine-tuning on two-dimensional checkerboard, image-brightness and $4\times$ super-resolution inverse problems. On super-resolution, our method achieves the lowest MMD among all evaluated baselines, including VFM. However, we find that it is not competitive for image inpainting, and we provide a detailed analysis of its failure.

1.4 Thesis Overview

Chapter 2

Background

This chapter introduces the background required to understand the proposed method. We first formalise the goals of generative modelling and conditional generative modelling. We then introduce inverse problems as the conditional generation setting considered in this thesis and relate them to the broader framework of reward alignment. Next, we review the flow-based generative models on which the project builds upon: normalising flows, flow models trained through flow matching, and flow maps. Finally, we introduce variational inference and Variational Flow Maps (VFM), which our approach directly builds on.

2.1 Generative Modelling

Suppose we wish to generate examples of a particular class of object. This class may be defined as broadly or narrowly as we like. We restrict our attention to objects that can be represented as vectors $\mathbf{x} \in \mathbb{R}^D$. In generative modelling, we assume that there exists a target probability distribution $p_{\text{target}}(\mathbf{x})$ which places high density on vectors corresponding to plausible examples of the specified class of object. The goal of generative modelling is to learn an algorithm that can produce samples from this target distribution.

Often, the target distribution is a data distribution $p_{\text{data}}(\mathbf{x})$ over, for example, images, videos or molecular structures. In this setting, we typically have access to a finite dataset $\mathcal{D} = \{\mathbf{x}^{(i)}\}_{i=1}^N$ of samples from the data distribution. Using this dataset, the aim is to learn a model distribution $p_{\theta}(\mathbf{x})$ that approximates $p_{\text{data}}(\mathbf{x})$. Depending on the generative modelling algorithm, it may support sampling from $p_{\theta}(\mathbf{x})$, evaluating its density, or both.

Many generative models, such as generative adversarial networks (GANs)

[29] and the flow-based generative models [4, 14, 27] considered in this thesis, generate data by first drawing noise $\mathbf{z} \sim p_{\text{base}}$ from a simple base distribution and then transforming it into a data sample $\mathbf{x}$. The learned transformation should induce a model distribution $p_{\theta}(\mathbf{x})$ that approximates the target distribution.

2.1.1 Conditional Generative Modelling

Often, we do not wish to generate an arbitrary sample from the target distribution. Instead, we wish to generate samples that satisfy a given condition $\mathbf{y} \in \mathbb{R}^M$, where $\mathbf{y} \sim p_Y(\mathbf{y})$. For example, p_Y may be a distribution over class labels, or text prompts. This task is known as conditional generative modelling. For a given condition $\mathbf{y}$, the task can be formalised as sampling from the conditional target distribution $p_{\text{target}}(\mathbf{x} \mid \mathbf{y})$ [10].

Typically, we do not want an algorithm that can generate samples for only one fixed condition. Instead, the aim is to learn an algorithm that can generate samples for different conditions $\mathbf{y} \sim p_Y(\mathbf{y})$. For example, an image generator may be required to accept different text prompts.

2.1.2 Inverse Problems

Inverse problem solving is a canonical example of conditional generation and is the focus of this thesis. To define an inverse problem, we first define the corresponding forward problem. Let $\mathbf{x} \sim p_{\text{data}}(\mathbf{x})$ be a data sample and let $\mathcal{A} : \mathbb{R}^D \rightarrow \mathbb{R}^M$ be a known forward operator. The forward operator acts on $\mathbf{x}$ to produce an observation

$$\mathbf{y} = \mathcal{A}(\mathbf{x}) + \boldsymbol{\eta}, \quad \boldsymbol{\eta} \sim \mathcal{N}(\mathbf{0}, \sigma_y^2 \mathbf{I}_M) \quad (2.1)$$

where $\boldsymbol{\eta}$ represents measurement noise. In image inverse problems, for example, $\mathcal{A}$ may blur, downsample or mask an image. The forward problem consists of computing the observation $\mathbf{y}$ from the data sample $\mathbf{x}$ [17].

The inverse problem is the reverse problem. Given an observation $\mathbf{y}$, the aim is to infer the unknown data sample $\mathbf{x}$ from which it was produced. However, the forward operator typically destroys information and is therefore not invertible. Several data samples may consequently be consistent with the same observation. Therefore, bayesian inverse problem solving instead aims to draw samples from

the posterior distribution

$$p_{\text{data}}(\mathbf{x} \mid \mathbf{y}) = \frac{p(\mathbf{y} \mid \mathbf{x})p_{\text{data}}(\mathbf{x})}{p_Y(\mathbf{y})} \quad (2.2)$$

where $p(\mathbf{y} \mid \mathbf{x})$ is the likelihood of observing $\mathbf{y}$ given $\mathbf{x}$. Under the Gaussian measurement model in Equation 2.1, the likelihood is

$$p(\mathbf{y} \mid \mathbf{x}) = \mathcal{N}(\mathbf{y} \mid \mathcal{A}(\mathbf{x}), \sigma_y^2 \mathbf{I}_M) \propto \exp\left(-\frac{\|\mathbf{y} - \mathcal{A}(\mathbf{x})\|_2^2}{2\sigma_y^2}\right) \quad (2.3)$$

Substituting the likelihood into Equation 2.2 gives

$$p_{\text{data}}(\mathbf{x} \mid \mathbf{y}) \propto p_{\text{data}}(\mathbf{x}) \exp\left(-\frac{\|\mathbf{y} - \mathcal{A}(\mathbf{x})\|_2^2}{2\sigma_y^2}\right) \quad (2.4)$$

The posterior therefore assigns high density to samples that are both plausible under $p_{\text{data}}(\mathbf{x})$ and consistent with the observation $\mathbf{y}$.

2.1.3 Relation to Reward Alignment

Conditional generation can be viewed as a particular instance of the broader task of reward alignment. This connection allows us to discuss general approaches for reward alignment in the related-work chapter with the view that they can be applied to conditional generation tasks, such as inverse problem solving.

Reward alignment aims to generate samples from the target distribution with specified desirable properties, as quantified by a reward function $r(\mathbf{x})$. For example, a reward function may compute an aesthetic rating of an image. Reward alignment can be formalised as sampling from

$$p_{\text{reward}}(\mathbf{x}) \propto p_{\text{target}}(\mathbf{x}) \exp(r(\mathbf{x})) \quad (2.5)$$

This distribution reweights the target distribution towards samples with higher reward, and is known as the reward-tilted distribution [30–32].

To express conditional generation as reward alignment, we let each condition $\mathbf{y}$ define a corresponding reward function $r_{\mathbf{y}}(\mathbf{x})$. In particular, we choose this reward to be the log-likelihood of the condition given the sample $r_{\mathbf{y}}(\mathbf{x}) = \log p(\mathbf{y} \mid \mathbf{x})$. The corresponding reward-tilted distribution is

$$p_{\text{reward}}(\mathbf{x} \mid \mathbf{y}) \propto p_{\text{target}}(\mathbf{x})p(\mathbf{y} \mid \mathbf{x}) \propto p_{\text{target}}(\mathbf{x} \mid \mathbf{y}) \quad (2.6)$$

For the inverse problems defined above, $p(\mathbf{y} \mid \mathbf{x})$ is the likelihood in Equation 2.3. In this example, samples receive a high reward when they are consistent with the observation. Inverse problem solving is therefore a particular instance of

conditional generation, which can itself be formulated as reward alignment [24].

2.2 Flow-Based Generative Modelling

Flow-based generative models transform samples from a tractable base distribution $p_{\text{base}}(\mathbf{z})$ into samples from a target distribution $p_{\text{target}}(\mathbf{x})$. Normalising flows represent this transformation as a finite composition of invertible maps [27]. Flow models and flow maps represent it as a continuous transformation governed by an ODE. While flow models learn the vector field defining the ODE [4, 10], flow maps instead aim to directly learn the integral of the ODE [33].

These models play different roles in this thesis. A pretrained flow map acts as the generative model that maps noise to data. Whereas we use a normalising flow to learn a conditional distribution over the noise space of the flow model (i.e. the noise adapter). Flow models provide the continuous-time transport framework from which flow maps are defined.

2.2.1 Normalising Flows

We first define normalising flows and explain how their densities are evaluated. We then introduce autoregressive flows, which are the specific variation of normalising flows that we use to implement the noise adapter. Finally we explain how normalising flows can be learned using forward or reverse KL divergence objectives.

Definition and Change of Variables

Suppose we wish to model a target distribution $p_{\text{target}}(\mathbf{x})$. A normalising flow begins with a simple base distribution $p_{\text{base}}(\mathbf{z})$ from which we can easily sample and whose density can be evaluated exactly. It then learns a transformation $T_\phi : \mathbb{R}^D \rightarrow \mathbb{R}^D$ that maps samples from this base distribution to samples whose distribution approximates the target distribution. More precisely, samples are generated according to

$$\mathbf{z} \sim p_{\text{base}}(\mathbf{z}) \quad \mathbf{x} = T_\phi(\mathbf{z}) \quad \mathbf{x} \sim p_\phi(\mathbf{x}) \quad (2.7)$$

and the parameters ϕ of the transformation are learned such that $p_\phi(\mathbf{x})$ approximates $p_{\text{target}}(\mathbf{x})$ [27].

The defining property of a normalising flow is that T_ϕ is invertible and

continuously differentiable, and that its inverse is also continuously differentiable. Such a transformation is known as a diffeomorphism. Invertibility allows samples to be mapped in both directions between the base and model spaces. Given $\mathbf{z} \sim p_{\text{base}}$, a sample from p_ϕ is generated by evaluating the forward transformation $\mathbf{x} = T_\phi(\mathbf{z})$. Conversely, a data point $\mathbf{x}$ can be mapped to its unique corresponding base point $\mathbf{z} = T_\phi^{-1}(\mathbf{x})$.

Because T_ϕ is a diffeomorphism, the density of p_ϕ is well-defined and can be evaluated using the change-of-variables formula

$$p_\phi(\mathbf{x}) = p_{\text{base}}\left(T_\phi^{-1}(\mathbf{x})\right) \left| \det \frac{\partial T_\phi^{-1}(\mathbf{x})}{\partial \mathbf{x}} \right| \quad (2.8)$$

To evaluate the density according to Equation 2.8, the inverse transformation must not only exist but also be computationally tractable. Intuitively, Equation 2.8 states that in order to compute the density of $\mathbf{x} \sim p_\phi$, first consider mapping $\mathbf{x}$ back to its corresponding base point $\mathbf{z} = T_\phi^{-1}(\mathbf{x})$. Subsequently, the density at $\mathbf{x}$ is equal to the base density at $\mathbf{z}$, weighted according to the local change in volume produced by T_ϕ . If T_ϕ expands the space around $\mathbf{z}$, the same probability mass is spread across a larger region and the density decreases. If it contracts the space, the density increases.

When $\mathbf{x}$ has been generated from a known base point $\mathbf{z}$, the same density can instead be evaluated using the Jacobian of the forward transformation

$$p_\phi(\mathbf{x}) = p_{\text{base}}(\mathbf{z}) \left| \det \frac{\partial T_\phi(\mathbf{z})}{\partial \mathbf{z}} \right|^{-1} \quad \mathbf{x} = T_\phi(\mathbf{z}) \quad (2.9)$$

This form avoids evaluating T_ϕ^{-1} because the corresponding base point $\mathbf{z}$ is already known, and will be useful when deriving the reverse-KL training objective later.

To summarise, a normalising flow supports both sampling and exact density evaluation. Sampling requires drawing $\mathbf{z} \sim p_{\text{base}}$ and evaluating $T_\phi(\mathbf{z})$. Evaluating the density at a given data point $\mathbf{x}$ requires computing $T_\phi^{-1}(\mathbf{x})$ and the corresponding Jacobian determinant. For an unrestricted $D \times D$ Jacobian, computing this determinant requires $\mathcal{O}(D^3)$ operations. Normalising-flow architectures must therefore choose the functional form of T_ϕ so that it remains expressive and invertible while its inverse and Jacobian determinant can be evaluated efficiently.

Constructing Expressive and Tractable Flows

In practice, the transformation T_ϕ is parameterised by a neural network. However, an arbitrary neural network is not guaranteed to be invertible or to have a

tractable Jacobian determinant. Restrictions must therefore be placed on the functional form of T_ϕ to ensure these properties are satisfied, without limiting its ability to represent a complex target distribution.

A common approach is to construct T_ϕ as a composition of L relatively simple invertible transformations, which we refer to as flow layers

$$T_\phi = T_{\phi_L} \circ T_{\phi_{L-1}} \circ \cdots \circ T_{\phi_1} \quad (2.10)$$

where $\phi = (\phi_1, \dots, \phi_L)$. Starting from $\mathbf{z}_0 = \mathbf{z}$, each flow layer is applied in turn according to $\mathbf{z}_\ell = T_{\phi_\ell}(\mathbf{z}_{\ell-1})$, with $\mathbf{z}_L = \mathbf{x}$. It can be proved that if every layer is a diffeomorphism, their composition is also a diffeomorphism. Furthermore, the inverse transformation T_ϕ^{-1} can be computed by applying the inverse of each layer in reverse order.

Given this composition 2.10, the overall Jacobian determinant of T_ϕ can be simply decomposed into the determinants of the individual layers. Let $J_{T_{\phi_\ell}}(\mathbf{z}_{\ell-1})$ denote the Jacobian of layer T_{ϕ_ℓ} evaluated at its input. The chain rule gives

$$\log |\det J_{T_\phi}(\mathbf{z})| = \sum_{\ell=1}^L \log |\det J_{T_{\phi_\ell}}(\mathbf{z}_{\ell-1})| \quad (2.11)$$

where we have taken the logarithm of the Jacobian determinant to convert the product of the layerwise determinants into a sum. Now, if the inverse and Jacobian determinant of each layer can be evaluated efficiently, the same is therefore true of the overall flow. At the same time, composing several simple layers can produce a significantly more expressive transformation than any individual layer, while the cost of evaluating the log-determinant grows only linearly with the number of layers [27].

Several approaches have been developed for constructing flow layers that are invertible, expressive and with cheap Jacobian determinant. These include autoregressive flows, linear flows and residual flows [27]. We focus on autoregressive flows because they have been shown to scale effectively to generative modelling for high-dimensional data, including images [34].

Autoregressive Flow Layers

Consider a single flow layer T_{ϕ_ℓ} . To simplify notation, denote its input by $\mathbf{z} \in \mathbb{R}^D$ and its output by $\mathbf{z}' = T_{\phi_\ell}(\mathbf{z}) \in \mathbb{R}^D$. An autoregressive flow layer splits the coordinates of $\mathbf{z}$ into K equal-sized blocks $\mathbf{z}_1, \dots, \mathbf{z}_K$. The output $\mathbf{z}'$ is split into corresponding blocks $\mathbf{z}'_1, \dots, \mathbf{z}'_K$. Each output block $\mathbf{z}'_i$ is obtained by applying an elementwise transformation to the corresponding input block $\mathbf{z}_i$.

The parameters of this transformation may depend only on the preceding input blocks $\mathbf{z}_{<i} = (\mathbf{z}_1, \dots, \mathbf{z}_{i-1})$. In particular,

$$\mathbf{z}'_i = \tau(\mathbf{z}_i; \mathbf{h}_i) \quad \mathbf{h}_i = c_i(\mathbf{z}_{<i}) \quad i = 1, \dots, K \quad (2.12)$$

The function τ , called the transformer, performs the elementwise transformation of $\mathbf{z}_i$. Its parameters $\mathbf{h}_i$ are predicted by the i th conditioner c_i , which is typically implemented using a neural network. Here, transformer refers to the function that transforms each block, rather than the Transformer neural-network architecture.

In order for the autoregressive flow layer is invertible, the transformer τ must be invertible with respect to $\mathbf{z}_i$ when its parameters $\mathbf{h}_i$ are fixed. This can be achieved by ensuring the transformer is a strictly monotonic function of $\mathbf{z}_i$. However, we do not require that the conditioner is invertible.

We now give an inductive argument for why the layer constructed above is invertible. Suppose that $\mathbf{z}'$ is given and we wish to recover $\mathbf{z}$. The parameters $\mathbf{h}_1 = c_1(\emptyset)$ can be computed immediately because they do not depend on any input blocks. The first input block can therefore be recovered as $\mathbf{z}_1 = \tau^{-1}(\mathbf{z}'_1; \mathbf{h}_1)$. Now assume that the first $i - 1$ input blocks $\mathbf{z}_{<i}$ have been recovered. These blocks can be used to compute the parameters $\mathbf{h}_i = c_i(\mathbf{z}_{<i})$. The i th input block can then be recovered as $\mathbf{z}_i = \tau^{-1}(\mathbf{z}'_i; \mathbf{h}_i)$. By induction, every input block can be recovered in this way. Therefore, T_{ϕ_ℓ} is invertible. Importantly, note that evaluating $T_{\phi_\ell}^{-1}$ must be performed iteratively in K sequential steps, first computing $\mathbf{z}_1$ before $\mathbf{z}_2$ and so on until $\mathbf{z}_K$. Each step requires evaluating both the conditioner and the transformer.

Having established that the layer is invertible, we now show that its Jacobian determinant can be evaluated efficiently. Since the transformer τ acts elementwise, each coordinate in the output block $\mathbf{z}'_i$ depends only on the corresponding coordinate in $\mathbf{z}_i$ (and on the preceding input blocks $\mathbf{z}_{<i}$). Importantly, each coordinate in the output $\mathbf{z}'_i$ does not depend on any later coordinates of the input. The Jacobian of the layer is therefore lower-triangular. Because the determinant of any triangular matrix is simply the product of its diagonal entries, the log-determinant of T_{ϕ_ℓ} can be computed in $\mathcal{O}(D)$ time [27].

Computational Direction: IAF and MAF

Recall that sampling from a normalising flow consists of applying each layer T_{ϕ_ℓ} in order, transforming the base sample $\mathbf{z}$ to $\mathbf{x}$. By contrast, evaluating the density of a given point $\mathbf{x}$ using the change-of-variables formula 2.8 requires applying

the inverse layers $T_{\phi_L}^{-1}, \dots, T_{\phi_1}^{-1}$. In the previous subsection, we showed that evaluating the inverse of a single autoregressive layer $T_{\phi_\ell}^{-1}$ requires K sequential steps.

We now show that the forward transformation $\mathbf{z}' = T_{\phi_\ell}(\mathbf{z})$ does not need to be evaluated sequentially. Given $\mathbf{z}$, all input blocks $\mathbf{z}_1, \dots, \mathbf{z}_K$ are already known. Since each $\mathbf{h}_i$ depends only on the input blocks $\mathbf{z}_{<i}$, all parameters $\mathbf{h}_1, \dots, \mathbf{h}_K$ can be computed simultaneously. In practice, a single neural network takes the complete vector $\mathbf{z}$ as input and produces all of these parameters in one forward pass. The connections of this neural network are masked to ensure that each output $\mathbf{h}_i$ cannot depend on the input blocks $\mathbf{z}_{\geq i}$ [35]. Once all parameters $\mathbf{h}_1, \dots, \mathbf{h}_K$ are known, every output block $\mathbf{z}'_i = \tau(\mathbf{z}_i; \mathbf{h}_i)$ can also be computed simultaneously. Therefore, all blocks of $\mathbf{z}' = T_{\phi_\ell}(\mathbf{z})$ can be evaluated in parallel, whereas the blocks of $\mathbf{z} = T_{\phi_\ell}^{-1}(\mathbf{z}')$ must be recovered sequentially.

In practice, this means that sampling, which evaluates $\mathbf{z}' = T_{\phi_\ell}(\mathbf{z})$, is comparatively inexpensive, whereas density evaluation, which requires $\mathbf{z} = T_{\phi_\ell}^{-1}(\mathbf{z}')$, is more expensive. If an application instead prioritises fast density evaluation, we can define the autoregressive flow layer such that the parameters $\mathbf{h}_i$ are computed as a function of the output blocks $\mathbf{z}'_{<i}$ rather than the input blocks $\mathbf{z}_{<i}$. In particular, we define $\mathbf{h}_i = c_i(\mathbf{z}'_{<i})$. This change preserves the invertibility and tractable lower-triangular Jacobian of the layer. However, computing all blocks of $\mathbf{z}' = T_{\phi_\ell}(\mathbf{z})$ now requires K sequential steps, whereas computing all blocks of $\mathbf{z} = T_{\phi_\ell}^{-1}(\mathbf{z}')$ can be parallelised.

When the parameters $\mathbf{h}_i$ are computed from the input blocks $\mathbf{z}_{<i}$, the layer is known as an inverse autoregressive flow (IAF) [36]. When they are instead computed from the output blocks $\mathbf{z}'_{<i}$, the layer is known as a masked autoregressive flow (MAF) [37]. IAF provides fast sampling, whereas MAF provides fast density evaluation.

Transformer Autoregressive Flows

We now introduce the normalising-flow architecture used to represent the noise adapter in this project: Transformer Autoregressive Flow (TarFlow) [34]. TarFlow constructs the overall normalising flow as a composition of L masked autoregressive flow layers. It adapts the original MAF architecture [37] to better model distributions over images. It achieves this through four main design choices. We explain each of these choices in turn.

The first design choice determines how the image-shaped variable $\mathbf{z}$ is divided

into the K blocks $\mathbf{z}_1, \dots, \mathbf{z}_K$. Let $\mathbf{z} \in \mathbb{R}^{C \times H \times W}$, where C , H and W denote the number of channels, height and width. Given a patch size P , TarFlow divides the $H \times W$ spatial grid into H/P rows and W/P columns of non-overlapping $P \times P$ patches. Each patch spans all C channels and therefore contains CP^2 coordinates. TarFlow flattens each patch to form a single block, which it refers to as a token. This produces $K = (H/P)(W/P)$ tokens, initially ordered by scanning the image from left to right and top to bottom.

The second design choice specifies the transformer τ applied to each token. TarFlow uses the elementwise affine transformation

$$\tau(\mathbf{z}_i; \mathbf{h}_i) = \exp(\boldsymbol{\alpha}_i) \odot \mathbf{z}_i + \boldsymbol{\beta}_i \quad \mathbf{h}_i = (\boldsymbol{\alpha}_i, \boldsymbol{\beta}_i) \quad (2.13)$$

where $\boldsymbol{\alpha}_i, \boldsymbol{\beta}_i \in \mathbb{R}^{CP^2}$ and $\odot$ denotes elementwise multiplication. The exponential ensures that every scale is positive, so the transformation is invertible.

The third design choice specifies the conditioner c_i . Recall that, in a masked autoregressive flow, the transformation parameters are computed via $\mathbf{h}_i = c_i(\mathbf{z}'_{<i})$. TarFlow uses a single causal Transformer neural network to compute $\mathbf{h}_1, \dots, \mathbf{h}_K$. Each token $\mathbf{z}'_i$ is first projected from dimension CP^2 to the hidden dimension of the Transformer, and a learned positional embedding is added. The resulting sequence is then processed by a stack of Transformer layers. Each layer contains a causal multi-head self-attention layer followed by a multilayer perceptron layer. The causal attention mask prevents each token from attending to later tokens. A final linear projection maps each token embedding to the affine parameters $(\boldsymbol{\alpha}_i, \boldsymbol{\beta}_i)$.

The fourth design choice concerns the token ordering. TarFlow reverses the order of the tokens between successive flow layers, so that tokens which appear early in one layer appear late in the next. Together, these design choices result in a scalable normalising flow architecture for modelling image distributions. In our image inverse problem experiments, we choose TarFlow to implement our noise adapter because the noise adapter models a distribution over the image-shaped noise space of a flow map.

Training Normalising Flows

So far, we have described how to construct a normalising flow T_ϕ that induces a model distribution $p_\phi(\mathbf{x})$. We now discuss how to learn the parameters ϕ such that $p_\phi(\mathbf{x})$ approximates the target distribution $p_{\text{target}}(\mathbf{x})$. A natural approach is to minimise the KL divergence between these distributions. Since the KL

divergence is asymmetric, we may minimise either the forward KL divergence $D_{\text{KL}}(p_{\text{target}} \| p_\phi)$ or the reverse KL divergence $D_{\text{KL}}(p_\phi \| p_{\text{target}})$.

We first define the forward KL loss as

$$\begin{aligned}
\mathcal{L}_{\text{fKL}}(\phi) &:= D_{\text{KL}}(p_{\text{target}} \| p_\phi) \\
&= \mathbb{E}_{\mathbf{x} \sim p_{\text{target}}} \left[\log \frac{p_{\text{target}}(\mathbf{x})}{p_\phi(\mathbf{x})} \right] \\
&= -\mathbb{E}_{\mathbf{x} \sim p_{\text{target}}} [\log p_\phi(\mathbf{x})] + \mathbb{E}_{\mathbf{x} \sim p_{\text{target}}} [\log p_{\text{target}}(\mathbf{x})] \\
&= -\mathbb{E}_{\mathbf{x} \sim p_{\text{target}}} [\log p_\phi(\mathbf{x})] + \text{const} \tag{2.14}
\end{aligned}$$

The final term is constant with respect to ϕ . Minimising $\mathcal{L}_{\text{fKL}}$ is therefore equivalent to maximising the expected log-likelihood assigned by the normalising flow to samples from the target distribution. This is precisely maximum-likelihood training. It requires samples from p_{target} and the ability to evaluate T_ϕ^{-1} and its Jacobian determinant through the change-of-variables formula 2.8. It does not require evaluating the density of p_{target} .

We next define the reverse KL loss as

$$\mathcal{L}_{\text{rKL}}(\phi) := D_{\text{KL}}(p_\phi \| p_{\text{target}}) = \mathbb{E}_{\mathbf{x} \sim p_\phi} [\log p_\phi(\mathbf{x}) - \log p_{\text{target}}(\mathbf{x})] \tag{2.15}$$

The expectation in Equation 2.15 is taken over $\mathbf{x} \sim p_\phi$. Since p_ϕ is obtained by transforming the base distribution through T_ϕ , the standard reparameterisation trick [38] allows this expectation to be rewritten over $\mathbf{z} \sim p_{\text{base}}$. Substituting the forward change-of-variables formula 2.9 then gives

$$\begin{aligned}
\mathcal{L}_{\text{rKL}}(\phi) &= \mathbb{E}_{\mathbf{z} \sim p_{\text{base}}} [\log p_\phi(T_\phi(\mathbf{z})) - \log p_{\text{target}}(T_\phi(\mathbf{z}))] \\
&= \mathbb{E}_{\mathbf{z} \sim p_{\text{base}}} [\log p_{\text{base}}(\mathbf{z}) - \log |\det J_{T_\phi}(\mathbf{z})| - \log p_{\text{target}}(T_\phi(\mathbf{z}))] \tag{2.16}
\end{aligned}$$

Equation 2.16 shows that reverse-KL training requires sampling from p_{base} , evaluating T_ϕ and its Jacobian determinant, and evaluating the target density at the generated sample. Unlike forward-KL training, it does not require samples from p_{target} . Furthermore, an unnormalised target density is sufficient because its unknown normalising constant is independent of ϕ and therefore does not affect optimisation.

Bibliography

- [1] Jonathan Ho, Ajay Jain, and Pieter Abbeel. *Denoising Diffusion Probabilistic Models*. 2020. arXiv: 2006.11239 [cs.LG]. URL: <https://arxiv.org/abs/2006.11239>.
- [2] Yang Song and Stefano Ermon. *Generative Modeling by Estimating Gradients of the Data Distribution*. 2020. arXiv: 1907.05600 [cs.LG]. URL: <https://arxiv.org/abs/1907.05600>.
- [3] Tero Karras et al. *Elucidating the Design Space of Diffusion-Based Generative Models*. 2022. arXiv: 2206.00364 [cs.CV]. URL: <https://arxiv.org/abs/2206.00364>.
- [4] Yaron Lipman et al. *Flow Matching for Generative Modeling*. 2023. arXiv: 2210.02747 [cs.LG]. URL: <https://arxiv.org/abs/2210.02747>.
- [5] Xingchao Liu, Chengyue Gong, and Qiang Liu. *Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow*. 2022. arXiv: 2209.03003 [cs.LG]. URL: <https://arxiv.org/abs/2209.03003>.
- [6] Robin Rombach et al. *High-Resolution Image Synthesis with Latent Diffusion Models*. 2022. arXiv: 2112.10752 [cs.CV]. URL: <https://arxiv.org/abs/2112.10752>.
- [7] Jonathan Ho et al. *Video Diffusion Models*. 2022. arXiv: 2204.03458 [cs.CV]. URL: <https://arxiv.org/abs/2204.03458>.
- [8] Haohe Liu et al. *AudioLDM: Text-to-Audio Generation with Latent Diffusion Models*. 2023. arXiv: 2301.12503 [cs.SD]. URL: <https://arxiv.org/abs/2301.12503>.
- [9] Michael S. Albergo, Nicholas M. Boffi, and Eric Vanden-Eijnden. *Stochastic Interpolants: A Unifying Framework for Flows and Diffusions*. 2025. arXiv: 2303.08797 [cs.LG]. URL: <https://arxiv.org/abs/2303.08797>.
- [10] Peter Holderrieth and Ezra Erives. *Introduction to Flow Matching and Diffusion Models*. 2026. arXiv: 2506.02070. URL: <https://diffusion.csail.mit.edu/>.

- [11] Nicholas M. Boffi, Michael S. Albergo, and Eric Vanden-Eijnden. *Flow map matching with stochastic interpolants: A mathematical framework for consistency models*. 2025. arXiv: 2406.07507 [cs.LG]. URL: <https://arxiv.org/abs/2406.07507>.
- [12] Nicholas M. Boffi, Michael S. Albergo, and Eric Vanden-Eijnden. *How to build a consistency model: Learning flow maps via self-distillation*. 2025. arXiv: 2505.18825 [cs.LG]. URL: <https://arxiv.org/abs/2505.18825>.
- [13] Yang Song et al. *Consistency Models*. 2023. arXiv: 2303.01469 [cs.LG]. URL: <https://arxiv.org/abs/2303.01469>.
- [14] Zhengyang Geng et al. *Mean Flows for One-step Generative Modeling*. 2025. arXiv: 2505.13447 [cs.LG]. URL: <https://arxiv.org/abs/2505.13447>.
- [15] Prafulla Dhariwal and Alex Nichol. *Diffusion Models Beat GANs on Image Synthesis*. 2021. arXiv: 2105.05233 [cs.LG]. URL: <https://arxiv.org/abs/2105.05233>.
- [16] Jonathan Ho and Tim Salimans. *Classifier-Free Diffusion Guidance*. 2022. arXiv: 2207.12598 [cs.LG]. URL: <https://arxiv.org/abs/2207.12598>.
- [17] Gregory Ongie et al. *Deep Learning Techniques for Inverse Problems in Imaging*. 2020. arXiv: 2005.06001 [eess.IV]. URL: <https://arxiv.org/abs/2005.06001>.
- [18] Chenlin Meng et al. *On Distillation of Guided Diffusion Models*. 2023. arXiv: 2210.03142 [cs.CV]. URL: <https://arxiv.org/abs/2210.03142>.
- [19] Huayu Chen et al. *Visual Generation Without Guidance*. 2025. arXiv: 2501.15420 [cs.CV]. URL: <https://arxiv.org/abs/2501.15420>.
- [20] Hyungjin Chung et al. *Diffusion Posterior Sampling for General Noisy Inverse Problems*. 2024. arXiv: 2209.14687 [stat.ML]. URL: <https://arxiv.org/abs/2209.14687>.
- [21] Jiaming Song et al. “Pseudoinverse-Guided Diffusion Models for Inverse Problems”. In: *International Conference on Learning Representations*. 2023. URL: https://openreview.net/forum?id=9_gsMA8MRKQ.
- [22] Bingliang Zhang et al. “Improving Diffusion Inverse Problem Solving with Decoupled Noise Annealing”. In: *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, June 2025, pp. 20895–20905. URL: <http://dx.doi.org/10.1109/CVPR52734.2025.01946>.
- [23] Jeongsol Kim, Bryan Sangwoo Kim, and Jong Chul Ye. *FlowDPS: Flow-Driven Posterior Sampling for Inverse Problems*. 2025. arXiv: 2503.08136 [cs.CV]. URL: <https://arxiv.org/abs/2503.08136>.

- [24] Abbas Mammadov et al. *Variational Flow Maps: Make Some Noise for One-Step Conditional Generation*. 2026. arXiv: 2603.07276 [cs.CV]. URL: <https://arxiv.org/abs/2603.07276>.
- [25] David M. Blei, Alp Kucukelbir, and Jon D. McAuliffe. “Variational Inference: A Review for Statisticians”. In: *Journal of the American Statistical Association* 112.518 (Apr. 2017), pp. 859–877. URL: <http://dx.doi.org/10.1080/01621459.2017.1285773>.
- [26] Danilo Jimenez Rezende and Shakir Mohamed. *Variational Inference with Normalizing Flows*. 2016. arXiv: 1505.05770 [stat.ML]. URL: <https://arxiv.org/abs/1505.05770>.
- [27] George Papamakarios et al. *Normalizing Flows for Probabilistic Modeling and Inference*. 2021. arXiv: 1912.02762 [stat.ML]. URL: <https://arxiv.org/abs/1912.02762>.
- [28] Frank Noé et al. *Boltzmann Generators – Sampling Equilibrium States of Many-Body Systems with Deep Learning*. 2019. arXiv: 1812.01729 [stat.ML]. URL: <https://arxiv.org/abs/1812.01729>.
- [29] Ian J. Goodfellow et al. *Generative Adversarial Networks*. 2014. arXiv: 1406.2661 [stat.ML]. URL: <https://arxiv.org/abs/1406.2661>.
- [30] Jerry Y. Huang et al. *How to Guide Your Flow: Few-Step Alignment via Flow Map Reward Guidance*. 2026. arXiv: 2604.27147 [cs.LG]. URL: <https://arxiv.org/abs/2604.27147>.
- [31] Peter Potapchik et al. *Meta Flow Maps enable scalable reward alignment*. 2026. arXiv: 2601.14430 [stat.ML]. URL: <https://arxiv.org/abs/2601.14430>.
- [32] Peter Holderrieth et al. *Diamond Maps: Efficient Reward Alignment via Stochastic Flow Maps*. 2026. arXiv: 2602.05993 [cs.LG]. URL: <https://arxiv.org/abs/2602.05993>.
- [33] Sander Dieleman. *Learning the integral of a diffusion model*. 2026. URL: <https://sander.ai/2026/05/06/flow-maps.html>.
- [34] Shuangfei Zhai et al. *Normalizing Flows are Capable Generative Models*. 2025. arXiv: 2412.06329 [cs.CV]. URL: <https://arxiv.org/abs/2412.06329>.
- [35] Mathieu Germain et al. *MADE: Masked Autoencoder for Distribution Estimation*. 2015. arXiv: 1502.03509 [cs.LG]. URL: <https://arxiv.org/abs/1502.03509>.
- [36] Diederik P. Kingma et al. *Improving Variational Inference with Inverse Autoregressive Flow*. 2017. arXiv: 1606.04934 [cs.LG]. URL: <https://arxiv.org/abs/1606.04934>.

- [37] George Papamakarios, Theo Pavlakou, and Iain Murray. *Masked Autoregressive Flow for Density Estimation*. 2018. arXiv: 1705.07057 [stat.ML]. URL: <https://arxiv.org/abs/1705.07057>.
- [38] Kevin P. Murphy. *Probabilistic Machine Learning: Advanced Topics*. MIT Press, 2023. URL: <http://probml.github.io/book2>.

Appendix A

Supplementary Material

A.1 Change of Variables

A.2 Score functions and score matching

A.3 Additional Experiments

A.3.1 GMM Experiments

Capacity Reward Hack in SR Inverse Problem